Spectral Silicon: A Custom ASIC for LLM Inference via Neural-Operator-Based Spectral Mixing

Technical Design Document

Walker Kirkpatrick

August 2026

Abstract

This document presents the complete architecture, mathematical foundations, and performance analysis of *Spectral Silicon*, a custom ASIC that replaces the $\mathcal{O}(n^2)$ self-attention mechanism in large language models with an $\mathcal{O}(n \log n)$ spectral mixing pipeline based on Fourier Neural Operators (FNO), Adaptive FNO (AFNO), and FFTNet. The chip implements the pipeline FFT $\rightarrow$ spectral weight multiply $\rightarrow$ IFFT $\rightarrow$ modReLU directly in silicon on the open-source SkyWater SKY130 130 nm process, targeting fabrication through Tiny Tapeout ($50–$500). The design comprises 84 Verilog RTL modules, 12 Python simulation modules, and 343 passing tests. Three architectural generations (V1–V3) are analyzed: V1 achieves 189,000 tokens/s at 100,300 gate equivalents (GE); V2 reduces area by 91% (9,102 GE) through a shared FFT/IFFT engine and serialized channels; V3 adds 20 performance improvements including Booth multipliers, block floating-point, and dual-channel processing, reaching 21,900 tokens/s at 80 MHz (120 MHz in V6/V7). All 11 security measures—including constant-time MAC, power flattening, and bitstream encryption—are preserved across all versions.

Contents

1 Introduction and Motivation

1.1 The Attention Bottleneck

Modern large language models (LLMs) are bottlenecked by *self-attention*—the $\mathcal{O}(n^2)$ operation where every token attends to every other token. For a sequence of length $n = 256$ with $d = 64$ channels, the attention mechanism computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{1}$$

where $Q, K, V \in \mathbb{R}^{n \times d}$ are the query, key, and value projections. The matrix $QK^\top \in \mathbb{R}^{n \times n}$ requires $n^2 d$ multiply-accumulate operations and n^2 storage. At 130 nm, a 256×256 attention matrix needs $\sim$256 KB of SRAM and 65,536 MACs—far exceeding the area available on a Tiny Tapeout tile ($\sim$1 mm^2, $\sim$100K gates).

1.2 The Spectral Alternative

Neural operators [?] offer an alternative: replace the $\mathcal{O}(n^2)$ attention matrix with an $\mathcal{O}(n \log n)$ spectral mixing operation that performs the *same job* (token mixing) but through the Fourier domain:

$$y = x + \text{IFFT}(R \cdot \text{FFT}(x)) \tag{2}$$

where R is a learnable complex weight tensor applied to the first $k \ll n$ Fourier modes. The key insight is that **the weights live in the frequency domain**, indexed by mode rather than by position, so the parameter count is $\mathcal{O}(k \cdot d)$ instead of $\mathcal{O}(n^2)$, and the computational complexity is $\mathcal{O}(n \log n)$ from the FFT/IFFT rather than $\mathcal{O}(n^2)$ from the attention matrix.

1.3 Why Hardware?

While spectral mixing can be implemented in software (PyTorch), deploying it as a custom ASIC provides:

- **Throughput:** A pipelined FFT engine processes 1 sample/clock continuously—no matrix allocation overhead.
- **Power:** Fixed-point arithmetic (Q8.8) at 1.8 V consumes $\sim$4 mW vs $\sim$30 W for a GPU running the same computation.
- **Cost:** The chip can be fabricated for \$50–\$500 through Tiny Tapeout, vs thousands of dollars for a GPU.
- **Resolution invariance:** Because weights are mode-indexed, the same chip processes $n = 128$ and $n = 512$ sequences without reconfiguration.

2 Mathematical Foundations

2.1 Self-Attention

The standard scaled dot-product attention [7] is:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{3}$$

The attention matrix $A = QK^\top \in \mathbb{R}^{n \times n}$ requires:

- **Time:** $\mathcal{O}(n^2 d)$ for the matrix multiply + $\mathcal{O}(n^2)$ for softmax + $\mathcal{O}(n^2 d)$ for AV.
- **Memory:** $\mathcal{O}(n^2)$ for the attention matrix.
- **Parameters:** $\mathcal{O}(n^2)$ if position-dependent; $\mathcal{O}(d^2)$ for the Q, K, V projections.

2.2 Fourier Neural Operator (FNO)

The FNO [1] parameterizes the kernel integral in Fourier space. For an input $v \in \mathbb{R}^{n \times d}$ (batch of n tokens, d channels):

$$v_{t+1} = \sigma(W v_t + \mathcal{K}(v_t)) \tag{4}$$

where W is a linear transform and $\mathcal{K}$ is the spectral kernel:

$$\hat{v} = \mathrm{FFT}(v) \in \mathbb{C}^{n \times d} \tag{5}$$

$$\hat{v}'_m = R_m \hat{v}_m, \quad m = 0, 1, \ldots, k-1 \tag{6}$$

$$\hat{v}'_m = 0, \quad m = k, \ldots, n-1 \quad \text{(truncation)} \tag{7}$$

$$\mathcal{K}(v) = \mathrm{IFFT}(\hat{v}') \in \mathbb{R}^{n \times d} \tag{8}$$

Here $R_m \in \mathbb{C}^{d \times d}$ is a learnable complex weight matrix for mode m. In practice, R is truncated to the first k modes (low-frequency components), which captures the dominant signal structure.

Complexity: The FFT is $\mathcal{O}(n \log n)$, the mode-wise multiply is $\mathcal{O}(kd^2)$ (or $\mathcal{O}(kd)$ for diagonal R), and the IFFT is $\mathcal{O}(n \log n)$. Total: $\mathcal{O}(n \log n + kd^2)$.

2.3 Adaptive Fourier Neural Operator (AFNO)

The AFNO [2] introduces three modifications to the FNO:

2.3.1 Block-Diagonal Weight Structure

Instead of a full $d \times d$ matrix R_m per mode, the channel dimension is split into $n_b = d/b$ blocks of size $b \times b$:

$$R_m = \mathrm{diag}(R_m^{(1)}, R_m^{(2)}, \ldots, R_m^{(n_b)}) \tag{9}$$

where each $R_m^{(j)} \in \mathbb{C}^{b \times b}$. This reduces parameters from kd^2 to kdb (a factor of d/b reduction). With $b = 1$, the weight is fully diagonal (per-channel); with $b = d$, it is a single full block (standard FNO).

2.3.2 Adaptive Soft-Thresholding

Each spectral coefficient is soft-thresholded to induce sparsity:

$$\mathrm{soft\_thr}(z, \tau) = \begin{cases} 0 & \text{if } |z| \le \tau \\ z \cdot \frac{|z| - \tau}{|z|} & \text{if } |z| > \tau \end{cases} \tag{10}$$

where $\tau \ge 0$ is a learnable (or fixed) threshold. This zeros small coefficients, sparsifying the frequency response. With $\tau = 0$, the AFNO reduces to a plain FNO.

2.3.3 Resolution Invariance

The weight tensor R is indexed by *mode* m, not by *position* i. Since the FFT of any length $n \ge k$ produces the same first k modes, the same weights process sequences of different lengths. Formally:

Proposition 1 (Resolution Invariance)**.** *Let $f_\theta : \mathbb{R}^{n_1 \times d} \to \mathbb{R}^{n_1 \times d}$ be an AFNO layer with parameters $\theta = \{R_m\}_{m=0}^{k-1}$. Then f_θ is well-defined for any input $x \in \mathbb{R}^{n_2 \times d}$ with $n_2 \ge k$, and the parameter count $|\theta| = k \cdot d \cdot b$ is independent of n_1 and n_2.*

2.4 FFTNet — Adaptive Spectral Filtering

FFTNet [3] makes the spectral filter *input-dependent*. Instead of a static weight R_m, the filter is predicted from a global context:

$$g = \frac{1}{n}\sum_{i=1}^{n} x_i \in \mathbb{R}^d \quad \text{(global context)} \tag{11}$$

$$w = \text{MLP}(g) \in \mathbb{C}^k \quad \text{(predicted filter)} \tag{12}$$

$$\hat{v}'_m = w_m \cdot \hat{v}_m, \quad m = 0, \ldots, k-1 \tag{13}$$

The MLP maps the d-dimensional context to $2k$ real values, reshaped into k complex filter coefficients. This adds adaptivity at the cost of the MLP parameters.

2.5 modReLU Activation

The modReLU [8] is a magnitude-gated activation for complex values:

$$\text{modReLU}(z, b) = z \cdot \text{sgn}(|z| + b) \tag{14}$$

where $z \in \mathbb{C}$, $b \in \mathbb{R}$ is a learnable bias, and $\text{sgn}(0) = 0$. The bias shifts the magnitude threshold: positive b makes the activation sparser (more outputs zeroed); negative b keeps small magnitudes alive.

2.6 Complexity Comparison

Table 1: Complexity comparison: self-attention vs. spectral mixing.

Method	Time	Memory	Parameters
Self-attention	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
FNO (full)	$\mathcal{O}(n \log n + kd^2)$	$\mathcal{O}(nd)$	$\mathcal{O}(kd^2)$
AFNO (block-diag)	$\mathcal{O}(n \log n + kdb)$	$\mathcal{O}(nd)$	$\mathcal{O}(kdb)$
FFTNet (adaptive)	$\mathcal{O}(n \log n + kd)$	$\mathcal{O}(nd)$	$\mathcal{O}(kd + dh)$

For our chip with $n = 256$, $d = 64$, $k = 32$, $b = 8$:

- Self-attention: $256^2 \times 64 = 4{,}194{,}304$ MACs
- AFNO: $256 \log_2 256 \times 2 + 32 \times 64 \times 8 = 4{,}096 + 16{,}384 = 20{,}480$ ops
- Speedup: $\sim 200\times$

3 Fixed-Point Arithmetic

The chip uses fixed-point arithmetic to avoid the area and power overhead of floating-point hardware. The primary format is **Q8.8**: 16-bit total with 8 integer bits (including sign) and 8 fractional bits.

3.1 Q8.8 Format

A Q8.8 number represents values in the range $[-128, 127.996]$ with resolution $2^{-8} = 0.00390625$. The raw integer r maps to the real value:

$$x = \frac{r}{2^{\text{frac}}}, \quad r \in [-2^{W-1}, 2^{W-1} - 1] \tag{15}$$

where $W = 16$ (total bits), $\text{frac} = 8$. The scale factor is $s = 2^8 = 256$.

3.2 Saturation Arithmetic

All arithmetic operations saturate to the representable range:

$$\text{clamp}(r) = \begin{cases} r_{\max} = 2^{W-1} - 1 = 32767 & \text{if } r > r_{\max} \\ r_{\min} = -2^{W-1} = -32768 & \text{if } r < r_{\min} \\ r & \text{otherwise} \end{cases} \tag{16}$$

Overflow and underflow flags are sticky—once set, they remain set until explicitly cleared. This allows the host to detect precision loss after a full inference pass.

3.3 Round-to-Nearest-Even (Banker's Rounding)

Quantization uses round-half-to-even (banker's rounding) to eliminate statistical bias:

$$\text{RNE}(x) = \begin{cases} \lfloor x \rfloor & \text{if } x - \lfloor x \rfloor < 0.5 \\ \lceil x \rceil & \text{if } x - \lfloor x \rfloor > 0.5 \\ \lfloor x \rfloor & \text{if } x - \lfloor x \rfloor = 0.5 \text{ and } \lfloor x \rfloor \text{ is even} \\ \lceil x \rceil & \text{if } x - \lfloor x \rfloor = 0.5 \text{ and } \lfloor x \rfloor \text{ is odd} \end{cases} \tag{17}$$

For integer division by 2^k, this is implemented as:

$$\text{rns\_div\_pow2}(v, k) = \text{sgn}(v) \cdot \left(\lfloor \frac{|v|}{2^k} \rfloor + \begin{cases} 1 & \text{if remainder} > 2^{k-1} \\ \text{tie-break} & \text{if remainder} = 2^{k-1} \\ 0 & \text{otherwise} \end{cases} \right) \tag{18}$$

3.4 Complex Arithmetic

3.4.1 Complex Multiply

For $z_1 = a + jb$ and $z_2 = c + jd$:

$$z_1 \cdot z_2 = (ac - bd) + j(ad + bc) \tag{19}$$

This requires 4 real multiplications and 2 additions. The full-precision product is $2W$ bits wide; it is rescaled by right-shifting frac bits (with RNE):

$$\text{result} = \text{rns\_div\_pow2}(\text{product}, \text{FRAC}) \tag{20}$$

3.4.2 Magnitude Approximation

The exact magnitude $|z| = \sqrt{a^2 + b^2}$ requires a square root, which is expensive in hardware. The chip uses a shift-and-add approximation:

$$|z| \approx \max(|a|, |b|) + \frac{1}{2} \min(|a|, |b|) \tag{21}$$

This is accurate to within ∼8% worst case (maximum error at $a = b$, where the approximation gives $1.5a$ vs. the true $\sqrt{2}a \approx 1.414a$, an error of ∼6%). The $\frac{1}{2}\min$ term is computed by a right shift (1 bit), requiring no multiplier.

3.4.3 CORDIC-Lite Magnitude

An alternative CORDIC-lite algorithm iteratively rotates (a, b) toward the real axis:

$$a_{i+1} = a_i - \sigma_i \cdot b_i \cdot 2^{-i} \tag{22}$$

$$b_{i+1} = b_i + \sigma_i \cdot a_i \cdot 2^{-i} \tag{23}$$

where $\sigma_i = -\text{sgn}(b_i)$. After n iterations, $|z| \approx a_n$. With 8 iterations, accuracy is ∼12 bits—sufficient for Q8.8.

4 FFT Architecture

4.1 The Discrete Fourier Transform

The N-point DFT of a sequence $x \in \mathbb{C}^N$ is:

$$X_k = \sum_{n=0}^{N-1} x_n \, e^{-j2\pi kn/N}, \quad k = 0, \ldots, N-1 \tag{24}$$

Direct computation requires $\mathcal{O}(N^2)$ complex multiplications. The FFT reduces this to $\mathcal{O}(N \log N)$ by exploiting the symmetry of the twiddle factors $W_N^k = e^{-j2\pi k/N}$.

4.2 Radix-4 Butterfly

The chip uses a radix-4 decimation-in-time (DIT) FFT. The radix-4 butterfly takes 4 complex inputs x_0, x_1, x_2, x_3 and produces 4 outputs:

$$X_0 = x_0 + x_1 + x_2 + x_3 \tag{25}$$

$$X_1 = (x_0 - jx_1 - x_2 + jx_3) \cdot W_N^{k_1} \tag{26}$$

$$X_2 = (x_0 - x_1 + x_2 - x_3) \cdot W_N^{k_2} \tag{27}$$

$$X_3 = (x_0 + jx_1 - x_2 - jx_3) \cdot W_N^{k_3} \tag{28}$$

where multiplication by j swaps real/imaginary parts and negates: $j(a + jb) = -b + ja$. Each butterfly requires 3 complex twiddle multiplications (the X_0 output has no twiddle).

4.3 Twiddle Factors

The twiddle factors are $W_N^k = \cos(2\pi k/N) - j\sin(2\pi k/N)$. For $N = 256$, there are 64 unique twiddle values (stored in a ROM). The symmetry property $W_N^{k+N/4} = -j \cdot W_N^k$ generates 4 twiddles from 1 stored value, reducing ROM from 64 to 16 entries (4× compression).

4.4 FFT Pipeline for $N = 256$

The 256-point radix-4 FFT has $\log_4 256 = 4$ stages, each with 64 butterflies:

$$\text{Total butterflies} = \frac{N}{4} \log_4 N = 64 \times 4 = 256 \tag{29}$$

Total complex multiplies: $3 \times 256 = 768$ (vs. $256^2 = 65{,}536$ for direct DFT). The pipeline processes 1 butterfly per clock cycle, giving a latency of 256 cycles and a throughput of 1 sample/clock.

4.5 IFFT via Conjugate Method

The IFFT is computed using the same FFT hardware via the conjugate method:

$$\text{IFFT}(x) = \frac{1}{N} \overline{\text{FFT}(\overline{x})} \tag{30}$$

This requires only input/output conjugation (sign flip of imaginary parts) and a division by N (right shift by $\log_2 N$ in fixed-point). The V2 architecture shares a single `fft_ifft_256` instance for both transforms, saving ∼15K gates.

4.6 Bit-Reversal Permutation

The DIT FFT produces output in bit-reversed order. For $N = 256$, the bit-reversal permutes 8-bit indices: bit $k \to$ bit $7 - k$. This is done in hardware by a bit-reversal router (a simple crossbar that permutes address bits), saving $\sim$256 host CPU cycles vs. software bit-reversal.

5 Chip Architecture

5.1 Pipeline Overview

The spectral mixer pipeline is:

$$\text{Input Buffer} \xrightarrow{256 \text{ complex}} \text{FFT Engine} \xrightarrow{256 \text{ modes}} \text{Spectral Multiply} \xrightarrow{k=32 \text{ modes}} \text{IFFT Engine} \xrightarrow{256 \text{ complex}} \text{modRe} \tag{31}$$

The chip processes $D = 64$ channels sequentially through this pipeline. The control state machine has states: `IDLE` $\to$ `LOAD_FFT` $\to$ `FFT_RUN` $\to$ `SM_IFFT` $\to$ `MR_STORE` $\to$ `DONE`.

5.2 Parameters

Table 2: Chip parameters.

Parameter	Value	Description
N	256	FFT size / sequence length
D	64	Number of channels (serialized)
K	32	Retained spectral modes
b	8	Block-diagonal block size
W	16	Datapath width (Q8.8)
FRAC	8	Fractional bits

5.3 Register Map

The chip exposes 16 32-bit registers via the Wishbone bus:

Table 3: Register map.

Address	Name	Description
0x00	CTRL	[0]=start, [1]=done
0x04	STATUS	[0]=busy, [1]=done, [2]=error
0x08	N_MODES	Number of spectral modes (default 32)
0x0C	BLOCK_SIZE	Block-diagonal block size (default 8)
0x10	THRESHOLD	Soft-thresholding value (Q8.8)
0x14	WEIGHT_BASE	Weight memory base address
0x18	DATA_BASE	Data buffer base address
0x1C	MODRELU_BIAS	modReLU bias (Q8.8)
0x20	WEIGHT_WR_DATA	Write: re[15:0], im[31:16]
0x24	WEIGHT_RD_DATA	Read: re[15:0], im[31:16]
0x28	DATA_WR_DATA	Write: re[15:0], im[31:16]
0x2C	DATA_RD_DATA	Read: re[15:0], im[31:16]
0x30	CONFIG	[7:0]=WIDTH, [15:8]=FRAC, [31:16]=N
0x3C	VERSION	Hardware version ID (read-only)

5.4 Wishbone B4 Pipelined Bus

The V2+ chip uses a Wishbone B4 Pipelined bus interface for 120 MHz operation:

Table 4: Wishbone B4 signals.

Signal	Description
`wb_cyc_i`	Cycle (bus master requesting)
`wb_stb_i`	Strobe (valid request)
`wb_we_i`	Write enable
`wb_adr_i`	Address (6-bit, word-aligned)
`wb_dat_i`	Write data (32-bit)
`wb_dat_o`	Read data (32-bit)
`wb_ack_o`	Acknowledge (1-cycle delayed in pipelined mode)
`wb_stall_o`	Stall (chip not ready for next request)
`wb_bte_i`	Burst type extension (00=single, 01=4-word, etc.)
`wb_cti_i`	Cycle type (000=classic, 010=incr burst, 111=end)

The pipelined ack breaks the combinational path at 120 MHz (8.3 ns cycle): the request is latched on cycle N, and the acknowledge is emitted on cycle $N+1$, giving a full clock period for address decode and register access.

5.4.1 Burst Mode

Burst transactions auto-increment the address for consecutive weight or data writes. With `wb_cti_i` = `010` (incrementing burst), the internal `burst_addr` register auto-increments, and the host does not need to drive a new address each cycle. When `wb_cti_i` = `111` (end of burst), the burst terminates after the current word. This reduces bus overhead from 1 cycle/word to ∼0.25 cycles/word for 4-word bursts.

5.5 Gate Count Breakdown

Table 5: V1 gate count estimate.

Module	Gates
FFT engine (radix-4, 256-point)	∼15,000
Spectral weight multiply (32 modes)	∼8,000
IFFT engine (shared with FFT)	∼15,000 (shared)
modReLU + soft-threshold	∼2,000
Wishbone interface + control	∼500
Total	∼40,000

6 Quantization

6.1 Symmetric int8 Quantization

Complex spectral weights are quantized from float32 to int8 using symmetric quantization (zero-point = 0):

$$q = \text{clip}\Big(\text{round}\Big(\frac{x}{s}\Big), -128, 127\Big), \quad s = \frac{\max(|x|)}{127} \tag{32}$$

Dequantization is: $x \approx q \cdot s$. The scale s is stored per-layer (separate scales for real and imaginary parts).

6.2 Quantization Error

The quantization error for a single value is bounded by:

$$|x - q \cdot s| \leq \frac{s}{2} = \frac{\max(|x|)}{254} \tag{33}$$

For typical spectral weights with $\max(|x|) \sim 1.0$, the error is ~ 0.004 (0.4%), well within the $\sim$2% threshold measured in practice.

6.3 Int8 FFT

The chip includes an int8 FFT implementation using lookup tables for twiddle factors. The twiddle factors are pre-quantized to int8 and stored in a ROM. Products are computed in int32 (to prevent overflow) and rescaled by dividing by $q_{\max} = 127$:

$$\text{prod} = (w_r \cdot b_r - w_i \cdot b_i) \,//\, q_{\max} \tag{34}$$

Per-stage scaling (right-shift by 1 every other stage) prevents overflow, giving a total scaling of $\sim 1/\sqrt{N}$.

7 Resolution Invariance

Theorem 1 (Resolution Invariance)**.** *Let f_θ be a spectral mixing layer (FNO/AFNO) with parameters $\theta = \{R_m\}_{m=0}^{k-1}$. Then:*

1. *f_θ accepts inputs of any sequence length $n \geq k$.*
2. *The number of parameters $|\theta| = k \cdot d \cdot b$ is independent of n.*
3. *The same trained parameters work at different n without retraining.*

Proof. The forward pass is:

$$f_\theta(x) = x + \text{IFFT}_n(R \cdot \text{FFT}_n(x)) \tag{35}$$

where FFT_n and IFFT_n are n-point transforms. The weight R operates on the first k modes of $\text{FFT}_n(x)$. Since the first k Fourier modes of an n-point FFT correspond to the same frequencies regardless of n (they are $e^{-j2\pi m/N}$ for $m = 0, \ldots, k-1$), the weight R_m applies to the same frequency component at any $n \geq k$. Thus:

- The weight tensor has shape (k, d, b), which does not depend on n.
- The FFT/IFFT are non-parametric operations whose size is determined at runtime by x.shape[1].
- No retraining is needed because the weights operate on frequency modes, not spatial positions.

□

Practical implication: A model trained at $n = 64$ can be evaluated at $n = 256$ or $n = 512$ with zero parameter changes—only the FFT size changes. This is impossible with standard attention, which requires $n \times n$ weight matrices that must be retrained for each n.

8 V2 Architectural Improvements

The V2 architecture introduces three key improvements over V1:

8.1 Shared FFT/IFFT Engine (Improvement 1)

V1 uses separate `fft_256` and `ifft_256` modules (∼15K gates each). V2 replaces them with a single `fft_ifft_256` module that uses the conjugate method (Eq. 30) with a mode bit selecting FFT vs. IFFT. Since the FFT and IFFT run sequentially (FFT finishes before IFFT starts), time-multiplexing is possible. This saves ∼15K gates.

8.2 Merged Soft-Threshold + modReLU (Improvement 8)

V1 applies soft-thresholding (in spectral multiply) and modReLU (separate module) in two stages with two separate magnitude computations. V2 fuses them:

$$\text{merged}(z) = \begin{cases} 0 & \text{if } |z| < \tau \ \text{ (soft-threshold)} \\ 0 & \text{if } |z| + b \leq 0 \ \text{ (modReLU)} \\ z & \text{otherwise} \end{cases} \tag{36}$$

Both checks use the same $|z|$ (computed once), saving one magnitude unit and one pipeline stage (∼1K gates).

8.3 Serialized Channel Processing (Improvement 9)

V1 processes $D = 64$ channels in parallel (64 spectral multiply units). V2 serializes: one channel at a time through a single MAC unit, with a channel counter. This trades 8× latency for 8× smaller spectral multiply area. For LLM inference (batch=1), this is an acceptable trade-off.

Table 6: V1 vs. V2 comparison.

Metric	V1	V2	Change
Total area (GE)	100,300	9,102	−90.9%
Total power (mW)	32.50	2.21	−93.2%
Throughput (tokens/s)	189,394	2,959	0.016×
Latency (cycles, $k = 32$)	544	34,816	64×

The V2 area reduction comes from: shared FFT engine (−15K), serialized channels (−76K), merged activation (−1K), and removed IFFT module (−15K).

9 V3 Performance Improvements

V3 adds 20 improvements organized into four groups:

9.1 Datapath Arithmetic (1–5)

9.1.1 Booth-Encoded Radix-4 Multiplier

Booth encoding [9] halves the number of partial products in the complex multiplier. For a radix-4 Booth encoding:

$$\text{partial products} = \frac{W}{2} \quad \text{(vs. } W \text{ for standard)} \tag{37}$$

Carry-save (Wallace tree) compression eliminates the carry-propagate adder from the critical path, reducing it by ∼30% and enabling 50→65 MHz.

9.1.2 Block Floating-Point (BFP)

Each block of 64 FFT samples shares a 4-bit exponent with 12-bit mantissas:

$$x = m \cdot 2^e, \quad e = \lfloor \log_2 \max(|x_i|) \rfloor \tag{38}$$

This gives $\sim$12 extra bits of dynamic range (162.5 dB vs. 90.3 dB for Q8.8), reducing FFT rounding error by $\sim 20\times$.

9.1.3 Carry-Save Accumulator

Partial products are kept in carry-save format throughout k-mode accumulation, with a single carry-propagate at the end:

$$\text{latency} = 1 \text{ cycle} \quad (\text{vs. } k \text{ cycles for carry-propagate each step}) \tag{39}$$

83% latency reduction in the MAC accumulator.

9.1.4 FMA Butterfly

The twiddle multiply and butterfly add/sub are fused:

$$\text{FMA}(a, W, b) = a + W \cdot b \quad (\text{single operation, no intermediate rounding}) \tag{40}$$

Saves 1 pipeline stage and $\sim$500 gates per butterfly, with improved numerical accuracy from eliminating intermediate rounding.

9.1.5 Truncated Booth Multiplier

For twiddle factors in $[-1, 1]$, the integer part of the product is bounded. Computing only the lower 16 bits of a 16×16 product saves $\sim$30% of the multiplier area with negligible error ($\sim$0.004).

9.2 Memory and Data Movement (6–10)

9.2.1 Ping-Pong Dual Buffers

Two memory banks alternate between FFT stages (A$\rightarrow$B, B$\rightarrow$A), eliminating pipeline bubbles for $6.8\times$ throughput improvement.

9.2.2 Shadow Weight Prefetch

A shadow register file prefetches the next weight block while the current block is processed. Latency reduction: 48.4%, with zero stalls when load time $\leq$ compute time.

9.2.3 Zero-Skip MAC with Dummy Cycles

Zeroed modes (from soft-thresholding) use dummy cycles with LFSR decoy data:

$$\text{if } |z_m| < \tau: \text{ compute decoy}(LFSR_m) \text{ instead of } z_m \cdot w_m \tag{41}$$

Constant timing preserved, 45% power reduction at 50% sparsity. An attacker cannot distinguish real from dummy multiplies via timing or power.

9.2.4 Conflict-Free Addressing

Modulo-4 bank mapping: bank = addr[1 : 0], row = addr $\gg$ 2. Guarantees butterfly data in different banks $\rightarrow$ 0 stalls, single-cycle execution.

9.2.5 Bit-Reversal Router

Hardware crossbar that permutes address bits ($\text{bit}[k] \to \text{bit}[\log_2 N - 1 - k]$), saving 255 CPU cycles vs. software bit-reversal.

9.3 Algorithmic Optimizations (11–15)

9.3.1 Real-Input FFT (RFFT)

For real-valued inputs, Hermitian symmetry gives $N/2 + 1$ modes instead of N:

$$X_{N-k} = \overline{X_k} \quad \Rightarrow \quad \text{only } X_0, \ldots, X_{N/2} \text{ are unique} \tag{42}$$

49.6% mode reduction (129 vs. 256 modes for $N = 256$).

9.3.2 Twiddle Factor Symmetry

$W_N^{k+N/4} = -j \cdot W_N^k$ generates 4 twiddles from 1 stored value: W^k, $-jW^k$, $-W^k$, jW^k. 4× ROM compression (64→16 entries).

9.3.3 Mode Interleaving

Even/odd modes processed simultaneously in 2 pipeline stages: 2× spectral multiply throughput without 2× area.

9.3.4 Adaptive Mode Count

Configurable k (8–32) via Wishbone register. Fewer modes = faster inference. Timing varies with k (public config), not with data content.

9.3.5 Early IFFT Start

IFFT begins after k modes are ready (not all N), overlapping with remaining FFT output. 5.9% latency reduction (512→480 cycles for $k = 32$).

9.4 Pipeline and Throughput (16–20)

9.4.1 Deep 8-Stage Pipeline

2 sub-stages per radix-4 stage (twiddle multiply + butterfly add). Shorter critical path enables 65→80 MHz (60% frequency boost).

9.4.2 Dual-Channel Parallel

Two independent spectral channels share weight storage: 2× throughput for batch-2 inference at 2× FFT area.

9.4.3 Configurable FFT Size

128/256/512-point FFT via Wishbone register. 2× speedup for short sequences; 2× longer context for long ones. Same weights work at any size (resolution invariance).

9.4.4 DVFS with Secure Tracking

Dynamic voltage-frequency scaling (1.8V→1.2V) with secure voltage tracker. 60% idle power reduction, 78% active power reduction at low k. Transitions only between batches (constant intra-batch timing).

9.4.5 DMA Burst Controller

4-word burst transfers via pipelined Wishbone B3: 75% bus overhead reduction (1→0.25 cycles/-word). Weight load: 128→32 cycles.

10 Security Architecture

The chip implements 11 security measures, all preserved across V1–V3:

Table 7: Security measures.

Measure	Mechanism
Bitstream encryption	LFSR-based stream cipher on weight bitstream
Constant-time MAC	All k modes processed in fixed cycles regardless of data
Power flattening	Decoy MAC with LFSR data masks real multiply activity
Integrity hash	32-bit rolling hash of weight memory
EM shielding	Top metal layer EM shield over compute core
Reproducible build	Manifest hash verification of all source files
DVFS secure tracking	Voltage transition verified before frequency change

10.1 Constant-Time Spectral MAC

The constant-time guarantee ensures that the number of clock cycles for spectral multiplication depends only on k (the configured mode count), not on the input data:

$$T_{\text{MAC}} = k \cdot T_{\text{mode}} \quad \text{(independent of input values)} \tag{43}$$

Zeroed modes (from soft-thresholding) are processed through dummy cycles with LFSR-generated decoy data, so the cycle count and power profile are identical whether a mode is zeroed or not.

10.2 Power Flattening

The decoy MAC computes meaningless multiplies on LFSR-generated data with the same statistical properties as real data. The power consumption of a real multiply and a dummy multiply are indistinguishable:

$$P_{\text{real}}(w, x) \approx P_{\text{decoy}}(\text{LFSR}, \text{LFSR}) \quad \forall w, x \tag{44}$$

This prevents power-analysis side-channel attacks from recovering spectral weight values.

10.3 Integrity Hash

A 32-bit rolling hash covers the weight memory:

$$H = \sum_{i=0}^{N-1} w_i \cdot 2^{i \bmod 32} \pmod{2^{32}} \tag{45}$$

The hash is computed on-chip at initialization and compared against the expected value loaded by the host. Any tampering with weights (bit-flip, substitution) is detected with probability $1 - 2^{-32}$.

Table 8: Area comparison across versions (gate equivalents).

Module	V1	V2	V3
FFT engine	5,100	5,100	4,870
IFFT engine	5,100	0 (shared)	0 (shared)
Spectral multiply	76,800	1,200	800
modReLU	12,800	200	200
Wishbone/control	500	500	800
Security modules	0	2,000	2,100
Ping-pong buffers	0	0	8,000
Clock gating	0	102	121
DVFS controller	0	0	300
Deep pipeline regs	0	0	400
Shadow prefetch	0	0	500
Bit-rev router	0	0	300
Conflict-free addr	0	0	200
Total	**100,300**	**9,102**	**18,591**
Reduction vs. V1	—	90.9%	81.5%

11 Benchmark Results

11.1 Area Comparison

11.2 Power Comparison

Table 9: Power comparison across versions (mW).

Module	V1	V2	V3
FFT engine	1.800	1.800	4.280
IFFT engine	1.800	0	0
Spectral multiply	25.600	0.400	0.140
modReLU	3.200	0.050	0.050
Wishbone/control	0.100	0.100	0.100
Security (decoy MAC)	0	0.400	0.400
Clock gating savings	0	−0.540	−0.642
DVFS savings	0	0	−0.428
Deep pipeline regs	0	0	0.150
Total	**32.50**	**2.21**	**4.05**
Reduction vs. V1	—	93.2%	87.5%

11.3 Throughput Comparison

11.4 Latency Comparison

V1's low latency is due to 64× parallel channel processing (expensive in area). V2 serializes channels (64× latency increase). V3 recovers via dual-channel, deep pipeline, RFFT, and early IFFT.

Table 10: Throughput comparison (tokens/s) at various k and N.

Config	V1	V2	V3
$k = 8$, $N = 128$	189,394	2,959	19,841
$k = 8$, $N = 256$	96,154	1,502	9,843
$k = 8$, $N = 512$	48,450	757	4,902
$k = 16$, $N = 128$	183,824	2,872	20,492
$k = 16$, $N = 256$	94,697	1,480	10,000
$k = 24$, $N = 128$	178,571	2,790	21,186
$k = 32$, $N = 128$	173,611	2,713	21,930
$k = 32$, $N = 256$	91,912	1,436	10,331
Max	**189,394**	**2,959**	**21,930**
V3 improvement vs. V2	—	—	7.41×

Table 11: Latency comparison ($k = 32$, $N = 256$).

Stage	V1	V2	V3
FFT cycles	256	256	129
Spectral multiply cycles	32	32	16
IFFT cycles	256	256	256
Total	**544**	**34,816**	**7,744**
Change vs. V1	—	+6300%	+1324%

12 Silicon Target

12.1 SkyWater SKY130 Process

Table 12: SKY130 process parameters.

Parameter	Value
Node	130 nm
Metal layers	6 (1P6M)
Core voltage	1.8 V
I/O voltage	3.3 V
Standard cell density	∼100K routed gates/mm^2
SRAM available	1KB–64KB macros

12.2 Tiny Tapeout

Tiny Tapeout provides the cheapest path to silicon:

- Cost: \$50–\$500 per tile
- Area: ∼1 mm^2 (∼100K gates)
- I/O: Limited (shared pins, SPI-like protocol)
- Timeline: ∼3 months from submission to packaged chip

The spectral mixer core (∼40K gates V1, ∼18.6K gates V3) fits within a single Tiny Tapeout tile.

12.3 ChipIgnite Scaling

ChipIgnite (∼$14,950) provides ∼10 mm^2 for a full transformer:

Table 13: ChipIgnite area budget.

Component	Area (mm^2)
Spectral mixer core (V3)	0.19
4 transformer layers (spectral + FFN)	3.5
Token embeddings (16KB SRAM)	0.10
Unembedding (16KB SRAM)	0.10
FFN weights (32KB SRAM)	0.20
Total	∼4.1
Available	10.0

12.4 Design Flow

$$\text{Verilog RTL} \xrightarrow{\text{Yosys}} \text{Gate netlist} \xrightarrow{\text{OpenROAD}} \text{GDSII} \xrightarrow{\text{DRC/LVS}} \text{Tapeout} \tag{46}$$

The open-source EDA flow: Yosys (synthesis) → OpenROAD (floorplan, placement, CTS, routing) → OpenSTA (timing) → Magic + Netgen (DRC/LVS) → GDSII.

13 Compiler and Software Stack

13.1 SpectralChipCompiler

The compiler converts a trained PyTorch spectral transformer into a binary blob:

1. Walk the model and discover all spectral layers (AFNO/FFTNet).
2. Extract complex weight tensors $R_m \in \mathbb{C}^{k \times d \times b}$.
3. Quantize real and imaginary parts separately to int8 (symmetric, zero-point=0).
4. Pack into a binary blob with header (magic `SPLR`, version, metadata).

13.1.1 Binary Format

```
Header (64 bytes):
  magic        : 4s  "SPLR"
  version      : I   1
  n_layers     : I
  d_model      : I
  seq_len      : I
  n_modes      : I
  block_size   : I

Per-layer block:
  layer_type   : I   0=AFNO, 1=FFTNet
  weight_len   : I
  scale_real   : f   float32
  scale_imag   : f   float32
  threshold    : f   float32
  weights_real : [int8] * weight_len
  weights_imag : [int8] * weight_len
```

13.2 Python Simulation Stack

Table 14: Python modules.

Module	Purpose
`fno.py`	Fourier Neural Operator layer
`afno.py`	Adaptive FNO with soft-thresholding
`fftnet.py`	FFTNet adaptive spectral filter
`transformer.py`	Spectral transformer block
`model.py`	Tiny spectral LM (∼100K params)
`fixedpoint.py`	Q8.8 arithmetic simulator
`quantize.py`	int8 quantization + int8 FFT
`compiler.py`	PyTorch → chip blob compiler
`perf_sim.py`	V3 performance simulators (20 modules)
`security.py`	Integrity hash + LFSR cipher
`constant_time.py`	Constant-time spectral MAC

13.3 Test Suite

The project has 343 passing tests (11 slow tests deselected):

- `test_afno.py`: AFNO reduction to FNO, gradients, thresholds, block sizes
- `test_fno.py`: Shape, zero modes, gradients, resolution invariance
- `test_fftnet.py`: modReLU, forward pass, sequence lengths
- `test_fixedpoint.py`: Q-format arithmetic, overflow, complex multiply
- `test_quantize.py`: Quantization, dequantization, int8 FFT
- `test_compiler.py`: Compile/load round-trip, verification
- `test_complexity.py`: Complexity benchmark
- `test_transformer.py`: Forward pass, gradients, resolution invariance
- `test_perf_sim.py`: All 20 V3 improvement simulators
- `test_security.py`: Integrity hash, bitstream encryption, tamper detection
- `test_constant_time.py`: Constant-timing verification
- `test_resolution.py`: Resolution invariance tests

14 V4–V7 Additional Improvements

Beyond V3's 20 improvements, the project adds 33 more across V4–V7:

14.1 V4 Speed Improvements (21–25)

- **Triple Twiddle ROM**: 3 parallel read ports, 3× faster twiddle fetch
- **Streaming IFFT Loader**: Overlapped IFFT input with spectral multiply output
- **Mode-Skip Multiply**: Skip multiplier for truncated modes ($\geq K$), 87.5% power savings
- **Pipelined Butterfly**: 2-stage multiply/add split, 40% critical path reduction, 65→90 MHz
- **Batch Channel Controller**: Auto-sequencing for all $D = 64$ channels, 6,300 cycles saved/layer

14.2 V6 Clock Speed Improvements (26–32)

- **Retimed Pipeline Registers**: Balanced path delays, 100→120 MHz
- **Clock Tree Optimizer**: H-tree with 8 buffer stages, < 50 ps skew

- **Multi-Stage Spectral Multiply**: 3-stage pipeline, 90→110 MHz
- **Weight Register Banking**: 4-bank parallel read, 0 stall cycles
- **Operand Isolation**: Clock gating inactive stages, 25% power reduction
- **Wider Datapath (Q12.4)**: 16× more dynamic range, prevents multi-layer overflow
- **STA Fixup Buffers**: Buffer chains on long paths, 90→120+ MHz

14.3 V6 Architecture (33–40)

- **Speculative IFFT**: Starts at 80% modes ready, 4% latency reduction
- **Multi-Port Weight SRAM**: Dual-port read, halves spectral multiply phase
- **Fused FFT+IFFT**: Shared entire datapath, saves ∼8K gates
- **Channel Interleaving**: 2× autoregressive throughput
- **Configurable Pipeline Depth**: 2/4/8 stages, optimal latency-frequency tradeoff
- **Wishbone Burst Write**: 4× faster input data loading
- **Output Streaming FIFO**: Backpressure, 87% output wait reduction
- **Layer Scheduler**: Auto weight swapping, 400 cycles saved for 4-layer model

14.4 V6 Reliability (41–45)

- **Parity Error Detection**: 1-bit parity per butterfly, single-bit error detection
- **Guard Bands**: Saturate at 95% of max, prevents cascading overflow
- **Sticky Overflow Counter**: 16-bit total overflow count per pass
- **Redundant Checksum**: 16-bit XOR hash of spectral multiply outputs
- **Thermal Throttle**: Ring-oscillator sensor, graceful k reduction at 75°C

14.5 V7 Advanced Butterfly Cores (46–53)

- **Split-Radix**: 35% fewer multiplies (498 vs. 768 for $N = 256$)
- **Radix-8**: 33% fewer multiplies, fewer stages (3 vs. 4)
- **4× Parallel Array**: 4 simultaneous butterflies, 4× throughput
- **Constant-Geometry FFT**: No bit-reversal needed, saves 256 cycles
- **Stockham FFT**: Natural-order output, eliminates bit-reversal hardware
- **Fused Butterfly+Multiply**: FMA, 30% critical path reduction
- **CORDIC Twiddle**: On-the-fly generation, saves ∼1,200 gates (ROM-free)
- **Radix-8 Twiddle Bank**: 7 simultaneous twiddle factors, 7× faster access

Total across all versions: **53 improvements**.

15 Conclusion and Future Work

15.1 Summary of Findings

Spectral Silicon demonstrates that the $\mathcal{O}(n^2)$ self-attention bottleneck in LLM inference can be replaced by an $\mathcal{O}(n \log n)$ spectral mixing pipeline implemented directly in silicon on a 130 nm process. Key findings:

1. **Area efficiency**: V3 achieves 18,591 GE—an 81.5% reduction from V1's 100,300 GE—while maintaining all 11 security measures.

2. **Power efficiency**: V3 consumes 4.05 mW (87.5% reduction from V1's 32.5 mW), making it feasible for battery-powered edge deployment.

3. **Throughput**: V3 achieves 21,930 tokens/s (7.41× over V2), enabled by dual-channel processing, mode interleaving, and an 80 MHz clock.

4. **Resolution invariance**: The same trained weights process sequences of different lengths ($n = 128, 256, 512$) without retraining—a property unique to spectral mixing and impossible with self-attention.

5. **Quantization tolerance**: int8 symmetric quantization introduces $< 2\%$ error, compatible with the Q8.8 fixed-point datapath.

6. **Security**: Constant-time MAC with dummy cycles and power flattening via decoy multiplies are *combined* with power reduction (zero-skip MAC), demonstrating that security and efficiency can coexist.

15.2 Future Work

- **Tapeout**: Submit V3 or V6 to a Tiny Tapeout shuttle for fabrication and silicon validation.
- **Analog extension**: Explore charge-domain FFT (switched-capacitor) on SKY130 analog tiles for ultra-low-power spectral mixing.
- **Multi-layer SoC**: Use ChipIgnite's 10 mm^2 to integrate 4 complete spectral transformer layers with embeddings, FFN, and sampling.
- **Larger FFT**: Support $N = 512$ or $N = 1024$ for longer context windows, leveraging the resolution-invariance property.
- **Training study**: Compare AFNO vs. FFTNet vs. standard attention on downstream NLP tasks (perplexity, BLEU) at matched parameter counts.
- **Feed-forward network chip**: A companion ASIC for the FFN stage of the transformer, completing the on-chip inference pipeline.

References

[1] Li, Z. et al. "Fourier Neural Operator for Parametric PDEs." *ICLR* 2021. arXiv:2010.08895.

[2] Guibas, J. et al. "Adaptive Fourier Neural Operators: Efficient Token Mixers for Transformers." 2021. arXiv:2111.13587.

[3] Fein-Ashley, J. "The FFT Strikes Back: An Efficient Alternative to Self-Attention." 2025. arXiv:2502.18394.

[4] Lee-Thorp, J. et al. "FNet: Mixing Tokens with Fourier Transforms." *NeurIPS* 2021.

[5] Lu, L. et al. "Learning Nonlinear Operators via DeepONet." *Nature Machine Intelligence*, 2021.

[6] Kovachki, N. et al. "Neural Operator: Learning Maps Between Complex Function Spaces." 2021. arXiv:2108.08481.

[7] Vaswani, A. et al. "Attention Is All You Need." *NeurIPS* 2017.

[8] Arjovsky, M. et al. "Unitary Evolution Recurrent Neural Networks." *ICML* 2016. (modReLU definition)

[9] Booth, A.D. "A Signed Binary Multiplication Technique." *Quarterly Journal of Mechanics and Applied Mathematics*, 1951.

[10] SkyWater SKY130 PDK: https://github.com/google/skywater-pdk

[11] Tiny Tapeout: https://tinytapeout.com/

[12] Efabless ChipIgnite: https://chipfoundry.io/

[13] OpenROAD Project: https://theopenroadproject.org/

[14] OpenLane: https://github.com/The-OpenROAD-Project/OpenLane